

shogiAI 數據科學專題報告

將棋棋譜資料建模、Policy/Value Network 與搜尋整合

專案：shogiAI

2026 年 5 月

摘要

本報告說明 shogiAI 專案中的數據科學部分。專案以 1000 份日本將棋 CSA 棋譜為資料來源，先經由 cshogi 解析與合法性檢查，再將棋局匯入 MySQL，最後從資料庫匯出可訓練的 policy/value dataset。模型任務包含兩個目標：第一，根據目前局面預測人類棋手的下一手；第二，根據目前下棋方視角估計該局面最後勝負。本專案使用 $43 \times 9 \times 9$ 的局面張量、13689 類走法標籤、小型 CNN policy/value network、AdamW 最佳化器、CosineAnnealingLR 學習率排程，並將模型推論結果整合到 alpha-beta 搜尋，用於 AI 對弈與候選手展示。報告重點包含資料來源、特徵工程、標籤設計、模型架構、訓練方法、評估指標、搜尋整合、限制與改進方向。

目录

1	專案在數據科學中的定位	2
2	資料來源與資料處理流程	2
2.1	資料來源	2
2.2	資料處理管線	3
3	問題定義	3
3.1	Policy Learning	3
3.2	Value Learning	4
4	特徵工程：局面編碼	4
4.1	為什麼需要特徵工程	4
4.2	棋盤棋子平面	4
4.3	持駒平面	5
4.4	手番平面	5
4.5	視角旋轉	5

目錄	2
5 標籤設計：走法編碼	5
5.1 一般移動	5
5.2 持駒打入	6
5.3 總走法空間	6
6 資料集內容	6
7 模型架構	7
7.1 整體架構	7
7.2 Feature Extractor	7
7.3 Policy Head	8
7.4 Value Head	8
8 訓練方法	8
8.1 資料切分	8
8.2 損失函數	9
8.3 最佳化器與學習率	9
9 評估指標	10
9.1 Policy 指標	10
9.2 Value 指標	10
9.3 目前結果	10
10 模型推論與搜尋整合	11
10.1 推論流程	11
10.2 合法手排序	11
10.3 與 Alpha-Beta 搜尋整合	11
11 數據科學上的限制	11
12 改進方向	12
13 結論	12
A 附錄：相關執行指令	12
A.1 匯出訓練資料	12
A.2 訓練模型	13
A.3 啟動含模型的 API	13

1 專案在數據科學中的定位

shogiAI 的數據科學目標不是單純訓練一個分類器，而是建立一條完整的棋類 AI 資料流程：

原始棋譜 → 合法局面資料 → 監督式學習樣本 → Policy/Value 模型 → 搜尋系統輔助

在棋類 AI 中，資料科學扮演三個角色：

1. **資料表示**：將人類棋譜中的文字格式轉換為可計算的張量。
2. **模式學習**：從歷史棋譜中學習人類在不同局面下的走法偏好與勝負趨勢。
3. **決策輔助**：將模型輸出用於排序合法手、估計局面價值，輔助傳統搜尋演算法。

因此本專案結合了資料工程、監督式學習、深度學習與傳統搜尋。資料科學部分的核心問題可表述為：

$$f_{\theta}(s) = (p, v)$$

其中 s 是目前將棋局面， p 是對所有可能走法的 policy 分數， v 是目前下棋方的勝負價值估計。

2 資料來源與資料處理流程

2.1 資料來源

本專案使用 data/ 目錄中的 CSA 棋譜作為原始資料。CSA 是日本將棋常見棋譜格式，可記錄：

- 棋手名稱與先後手。
- 賽事、日期、場地、開局等 metadata。
- 初始盤面。
- 每一手走法。
- 終局結果，例如投了、千日手、和棋。

目前專案資料規模如下：

項目	數值
CSA 棋譜數	1000 份
匯出訓練樣本數	111238 筆
局面張量形狀	(111238, 43, 9, 9)
走法標籤數	13689 類
已標記 value 樣本數	111238 筆

2.2 資料處理管線

專案的數據處理流程如下：

1. 使用 `src/import_csa_to_db.py` 解析 CSA 棋譜。
2. 使用 `cshogi.CSA.Parser` 取得棋局、走法與勝負結果。
3. 使用 `cshogi.Board` 從初始局面逐手重播。
4. 對每一手使用 `board.is_legal(move)` 檢查合法性。
5. 將棋局、棋手、走法與每手局面寫入 MySQL。
6. 使用 `src/export_policy_dataset.py` 從資料庫匯出模型資料。
7. 將每筆資料轉為 $(state, move, value)$ 形式。
8. 使用 `numpy.savez_compressed` 儲存成 `out/policy_dataset.npz`。

這個流程的關鍵是「訓練資料從資料庫匯出，而不是直接從原始檔案訓練」。這樣可確保資料已經經過一致的解析、清洗、合法性檢查與 metadata 整理，也方便日後用 SQL 篩選特定賽事、棋手或日期區間。

3 問題定義

3.1 Policy Learning

Policy learning 的任務是：給定目前局面 s_t ，預測實戰中的下一手 a_t 。

$$P_{\theta}(a_t \mid s_t)$$

這是一個多類別分類問題。每一筆訓練資料包含：

$$(s_t, a_t)$$

其中 s_t 是局面張量， a_t 是實戰走法的整數標籤。

3.2 Value Learning

Value learning 的任務是：給定目前局面 s_t ，預測目前下棋方最後是否會獲勝。

$$V_{\theta}(s_t) \in [-1, 1]$$

標籤定義為：

$$z_t = \begin{cases} 1, & \text{目前下棋方最後獲勝} \\ 0, & \text{和棋或千日手} \\ -1, & \text{目前下棋方最後落敗} \end{cases}$$

這種定義比直接預測「黑方勝率」更適合 negamax 搜尋，因為搜尋演算法本身也是從目前手番視角評分。

4 特徵工程：局面編碼

4.1 為什麼需要特徵工程

CSA、USI、SFEN 都是文字格式，神經網路無法直接使用。模型需要固定形狀的數值張量。本專案將每個局面編碼成：

$$state \in \mathbb{R}^{43 \times 9 \times 9}$$

其中 9x9 對應將棋棋盤，43 個 channel 對應不同棋子、持駒與手番資訊。

4.2 棋盤棋子平面

將棋棋子類型包含未升變與升變後棋子。本專案使用 14 種棋子類型：

- 步、香、桂、銀、金、角、飛、王。
- 成步、成香、成桂、成銀、馬、龍。

棋盤上的棋子分為己方與對方，因此：

$$14 \times 2 = 28$$

前 28 個平面用於表示棋盤上每格是否有某一類棋子。

4.3 持駒平面

將棋和西洋棋最大的不同之一是「持駒打入」。吃掉對手棋子後，棋子會變成自己的持駒，可在之後打回棋盤。因此局面表示必須包含持駒數量。

可打入的持駒類型有 7 種：

{步, 香, 桂, 銀, 金, 角, 飛}

本專案使用：

- 7 個平面表示己方持駒數量。
- 7 個平面表示對方持駒數量。

持駒平面不是 one-hot，而是整個 9×9 平面填入該棋子持有數量。這讓 CNN 在任何位置都能讀到持駒資訊。

4.4 手番平面

第 43 個平面用於補充輪到誰下或視角資訊。棋類模型必須知道手番，因為同一個盤面輪到不同方下，局面價值與合法手完全不同。

4.5 視角旋轉

專案預設使用 `orient_to_turn=True`。當輪到白方時，棋盤會旋轉 180 度，使模型總是從目前下棋方的角度觀察局面。

優點：

- 降低黑白方向差異。
- 增加資料對稱性。
- 讓相似攻防型態在模型眼中具有相同方向。

例如黑方的「向前推步」與白方的「向前推步」，若不旋轉，對模型而言方向相反；旋轉後都成為同一種局部模式。

5 標籤設計：走法編碼

5.1 一般移動

一般移動由三個資訊組成：

- 來源格：81 種。
- 目標格：81 種。
- 是否升變：2 種。

因此一般移動標籤數為：

$$81 \times 81 \times 2 = 13122$$

5.2 持駒打入

打入走法不需要來源格，而是由：

- 打入棋子種類：7 種。
- 目標格：81 種。

因此打入標籤數為：

$$7 \times 81 = 567$$

5.3 總走法空間

總 policy 類別數：

$$13122 + 567 = 13689$$

雖然任一局面中合法手通常遠少於 13689，但固定輸出空間使模型可以使用標準 cross entropy 訓練。

6 資料集內容

out/policy_dataset.npz 包含下列欄位：

欄位	說明
states	局面張量，形狀為 $(N, 43, 9, 9)$ 。
moves	實戰下一手的 policy label。
values	目前下棋方視角的勝負標籤。
value_masks	表示 value 標籤是否可信，避免未知結果影響 value loss。

欄位	說明
game_ids	每筆樣本所屬棋局，用於依棋局切分資料集。
meta	JSON 字串，包含 game id、ply、sfen、move usi、result 等資訊。
move_label_count	policy 類別總數，目前為 13689。
orient_to_turn	是否使用目前下棋方視角旋轉。

目前結果分布如下：

終局結果	樣本數
TORYO	110967
DRAW	104
SENNICHITE	167

7 模型架構

7.1 整體架構

模型定義於 `src/policy_model.py`，名稱為 `SmallPolicyValueNet`。其結構如下：

$$x \in \mathbb{R}^{43 \times 9 \times 9}$$

$$h = f_{\text{conv}}(x)$$

$$\hat{p} = f_{\text{policy}}(h), \quad \hat{p} \in \mathbb{R}^{13689}$$

$$\hat{v} = f_{\text{value}}(h), \quad \hat{v} \in [-1, 1]$$

7.2 Feature Extractor

Feature extractor 使用三層卷積：

```
Conv2d(43, 64, kernel_size=3, padding=1)
ReLU
Conv2d(64, 64, kernel_size=3, padding=1)
ReLU
Conv2d(64, 32, kernel_size=3, padding=1)
ReLU
```

卷積適合棋盤任務，因為棋子攻防、王周圍安全、升變區、棋子保護都和局部空間結構有關。

7.3 Policy Head

Policy head 將卷積特徵展平後輸出 13689 類：

```
Flatten
Linear(32 * 9 * 9, 512)
ReLU
Linear(512, 13689)
```

輸出是 logits，不直接經過 softmax。訓練時由 CrossEntropyLoss 內部處理 softmax；推論時只取合法手對應的 logits。

7.4 Value Head

Value head 輸出單一數值：

```
Flatten
Linear(32 * 9 * 9, 128)
ReLU
Linear(128, 1)
Tanh
```

最後使用 Tanh，使輸出範圍落在 $[-1, 1]$ ，符合勝負標籤定義。

8 訓練方法

8.1 資料切分

資料切分由 `split_by_game` 完成，以棋局為單位切分：

- 訓練集：約 80% 棋局。
- 驗證集：約 10% 棋局。
- 測試集：約 10% 棋局。

目前 checkpoint 中記錄：

集合	樣本數
Train	88473

集合	樣本數
Validation	11166
Test	11599

以棋局為單位切分是為了避免資料洩漏。若隨機切分每一筆樣本，同一盤棋的相鄰局面可能同時出現在訓練與測試中，使測試分數不真實。

8.2 損失函數

總損失：

$$\mathcal{L} = \mathcal{L}_{policy} + \lambda \mathcal{L}_{value}$$

其中 policy loss 使用 cross entropy：

$$\mathcal{L}_{policy} = -\log \frac{\exp(\hat{p}_a)}{\sum_j \exp(\hat{p}_j)}$$

value loss 使用 MSE：

$$\mathcal{L}_{value} = m(\hat{v} - z)^2$$

m 是 value mask， z 是勝負標籤， λ 對應 value-loss-weight，目前使用 0.25。

8.3 最佳化器與學習率

訓練使用：

- Optimizer：AdamW。
- Learning rate： 10^{-3} 。
- Weight decay： 10^{-4} 。
- Scheduler：CosineAnnealingLR。
- Epochs：12。

AdamW 比一般 Adam 更適合搭配 weight decay，可減少過度擬合。Cosine annealing 會使學習率逐步下降，讓訓練後期更新更穩定。

9 評估指標

9.1 Policy 指標

Policy 使用 Top-k accuracy：

- Top-1：模型最高分走法是否等於實戰走法。
- Top-3：實戰走法是否在模型前三名。
- Top-5：實戰走法是否在模型前五名。

棋類任務中 Top-5 很有意義，因為一個局面可能有多個合理手。模型不一定要完全猜中人類實戰手，只要能把合理候選手排在前面，就能輔助搜尋。

9.2 Value 指標

Value 使用：

- MSE loss。
- MAE。

MAE 可解釋為模型平均預測值與真實勝負標籤之間的距離。

9.3 目前結果

目前測試集結果：

指標	數值
Test policy loss	4.4340
Top-1 accuracy	0.2230
Top-3 accuracy	0.3695
Top-5 accuracy	0.4385
Value loss	1.0876
Value MAE	0.9263

結果顯示 policy model 已能在相當比例的局面中將實戰手排在前幾名，但 value head 仍有改進空間。原因是 value label 僅由終局勝負回推，中盤局面與終局勝負之間存在長期因果，學習難度較高。

10 模型推論與搜尋整合

10.1 推論流程

推論由 PolicyValuePredictor 處理：

1. 載入 out/policy_model.pt。
2. 讀取 checkpoint 中的 move_label_count 與 orient_to_turn。
3. 將 cshogi.Board 編碼成 state tensor。
4. 呼叫 PyTorch model 得到 logits 與 value。
5. 使用 zobrist hash 快取相同局面的推論結果。

10.2 合法手排序

模型輸出 13689 類，但實際下棋只能選合法手。推論時：

$$\text{score}(a) = \hat{p}_{\text{label}(a)} \quad \text{for } a \in A(s)$$

其中 $A(s)$ 是目前局面的合法手集合。接著只對合法手 logits 做 softmax，得到前端顯示的候選手機率。

10.3 與 Alpha-Beta 搜尋整合

AI 對弈不是直接選 policy 最大手，而是把 policy 用於 move ordering：

$$\text{legal moves} \rightarrow \text{policy rank} \rightarrow \text{alpha-beta search}$$

Alpha-beta 搜尋越早看到好手，越容易剪掉後續分支。因此 policy model 的主要作用是提升搜尋效率與候選手品質，而不是完全取代搜尋。

Value head 可作為根節點加權評估：

$$\text{score} = \text{handcrafted evaluation} + w \cdot V_{\theta}(s)$$

其中 w 由 --value-weight 控制。

11 數據科學上的限制

1. **資料量有限**：1000 盤棋可完成原型，但對強棋力深度模型而言仍偏少。
2. **Value label 較粗糙**：只由終局勝負回推，沒有中盤局面引擎分數。

3. **模型架構較小**：尚未使用 residual blocks、attention 或 transformer。
4. **未在訓練時使用合法手 mask**：目前 loss 對 13689 類完整空間計算，未直接限制非法手。
5. **資料分布不平衡**：大部分結果為投了，和棋樣本很少。

12 改進方向

- 增加棋譜數量，包含職業棋譜、公開對局與自我對弈資料。
- 加入合法手 mask loss，使模型更專注於合法候選手。
- 使用 residual CNN 或 transformer 架構提升模型表達能力。
- 使用強引擎為中盤局面產生 centipawn-like 評分，改善 value head。
- 針對不同手數階段建立 opening/middlegame/endgame 分析。
- 將 policy model 與搜尋結果記錄回資料庫，形成模型評估資料表。

13 結論

shogiAI 的數據科學部分展示了從原始棋譜到可互動 AI 的完整建模流程。專案並非只訓練模型，而是先建立資料庫與資料清洗流程，再設計局面張量、走法標籤、勝負標籤，最後用 CNN 學習 policy/value，並整合到 alpha-beta 搜尋。這種架構兼具資料可追蹤性、模型可訓練性與系統可展示性，是資料科學與傳統 AI 結合的完整範例。

A 附錄：相關執行指令

A.1 匯出訓練資料

```
python src\export_policy_dataset.py ^
--output out\policy_dataset.npz ^
--host 140.135.65.53 ^
--port 3306 ^
--user 11211213 ^
--password <password> ^
--database DB11211213
```

A.2 訓練模型

```
python src\train_policy.py ^  
  --input out\policy_dataset.npz ^  
  --output out\policy_model.pt ^  
  --batch-size 256 ^  
  --value-loss-weight 0.25
```

A.3 啟動含模型的 API

```
python src\csa_browser_api.py ^  
  --host 127.0.0.1 ^  
  --port 8000 ^  
  --source mysql ^  
  --db-host 140.135.65.53 ^  
  --db-port 3306 ^  
  --db-user 11211213 ^  
  --db-name DB11211213 ^  
  --policy-model out\policy_model.pt ^  
  --value-weight 200 ^  
  --policy-order-ply 1
```